

Kafka Segment Sizing, Throughput and Kafka Streams Guide

Understanding segment.bytes and segment.ms

In Kafka, segment.bytes and segment.ms control when a log segment is rolled (closed and a new segment is created).

1. segment.bytes

- Maximum size of a log segment file.
- When the segment reaches this size, Kafka creates a new segment.

Example:

segment.bytes=1073741824 (1 GB)

If messages arrive rapidly and the segment grows to 1 GB, Kafka rolls to a new segment immediately.

2. segment.ms

- Maximum time a segment can remain open.
- Even if the segment size is small, Kafka rolls it after this time.

Example:

segment.ms=604800000 (7 days)

If a topic receives very little traffic, Kafka will still create a new segment every 7 days.

Which One Wins?

Kafka rolls a segment when either condition is met first.

Scenario 1: High Traffic

segment.bytes=1GB

segment.ms=7 days

- 1 GB reached in 2 hours.
- New segment created after 2 hours.
- segment.bytes triggers first.

Scenario 2: Low Traffic

segment.bytes=1GB

segment.ms=7 days

- Only 100 MB written in 7 days.
- New segment created after 7 days.
- segment.ms triggers first.

Why Segment Rolling Matters

Retention and log cleanup work at the segment level, not individual messages.

Bad Example:

retention.ms=24 hours

segment.ms=7 days

A message may remain longer than 24 hours because Kafka cannot delete part of a segment.

Better Example:

segment.ms=1 hour

retention.ms=24 hours

Smaller segments allow Kafka to remove old data more accurately.

Typical Defaults

segment.bytes=1073741824 # 1 GB

segment.ms=604800000 # 7 days

Simple Rule

High-throughput topics -> segment.bytes usually triggers first.

Low-throughput topics -> segment.ms usually triggers first.

Kafka always rolls a segment when the first limit is reached.

Kafka Segment Sizing, Throughput and Kafka Streams Tuning Guide

Throughput Classification

Category	TPS	Approx Data Rate (1 KB Messages)
Very Low	<100	<100 KB/sec
Low	100-1,000	100 KB/sec - 1 MB/sec
Medium	1,000-50,000	1 MB/sec - 50 MB/sec
High	50,000-500,000	50 MB/sec - 500 MB/sec
Very High	>500,000	>500 MB/sec

Recommended Segment Settings

Throughput	TPS	segment.bytes	segment.ms	Reason
Very Low	<100	64 MB	30 min	Prevent long-open segments
Low	100-1K	128 MB	1 hour	Better retention cleanup
Medium	1K-50K	512 MB	1 day	Balanced file count and cleanup
High	50K-500K	1 GB	7 days	Size-based rolling dominates
Very High	>500K	1-2 GB	7 days	Reduce segment creation

Kafka Streams Internal Topics

Topic Type	segment.bytes	segment.ms	Reason
Repartition Topic	256 MB	1 hour	Faster recovery
Changelog Topic	256 MB	1 hour	Faster compaction
Window Store Changelog	128 MB	30 min	Faster cleanup
Session Store Changelog	128 MB	30 min	Efficient expiration
GlobalKTable Changelog	256 MB	1 hour	Faster rebuild
EOS Streams	256-512 MB	1 hour	Faster recovery

Cleanup Policy Recommendations

Cleanup Policy	segment.bytes	segment.ms
delete	512 MB - 1 GB	1 day - 7 days
compact	128 MB - 256 MB	1 hour
compact,delete	128 MB - 256 MB	30 min - 1 hour

Retention Recommendations

Retention	Recommended segment.ms
1 hour	5-15 min
1 day	30 min - 1 hour
7 days	1-24 hours
30 days	1 day
90+ days	1-7 days

Real-World Use Cases

Use Case	TPS	segment.bytes	segment.ms
User Registration	100	128 MB	1 hour
Payment Processing	5K	512 MB	1 day
Banking Transactions	10K	512 MB	1 day
Order Processing	20K	512 MB	1 day
CDC (Debezium)	10K-50K	512 MB-1 GB	1 day
Log Aggregation	100K+	1 GB	7 days
Clickstream Analytics	500K+	1-2 GB	7 days
IoT Sensors	1K-20K	128-512 MB	1 hour
Audit Events	1K	512 MB	1 day
Reference Data	Low	128 MB	1 hour
Kafka Streams Aggregations	10K-100K	256 MB	1 hour

How Quickly a 1 GB Segment Fills

100 TPS -> ~3 hours

1,000 TPS -> ~17 minutes

10,000 TPS -> ~1.7 minutes

100,000 TPS -> ~10 seconds

1,000,000 TPS -> ~1 second

Golden Rules

1. Event Topics: segment.bytes=1GB, segment.ms=1d-7d
2. Compacted Topics: segment.bytes=256MB, segment.ms=1h
3. Kafka Streams Internal Topics: segment.bytes=256MB, segment.ms=1h
4. Low-Traffic Topics: segment.bytes=128MB, segment.ms=1h

Large segments optimize throughput.

Small segments optimize retention, compaction, cleanup, and Kafka Streams state recovery.